

Engenharia Agêntica na Prática

Do Zero à Produção — o AI-Native Engineer constrói, sozinho, o que um time levou 4 anos para fazer.

Caso de uso **NorthWind Pay EDP** — uma fábrica agêntica que migra uma plataforma financeira legada para uma stack moderna, 100% open-source, reproduzível no laptop.

FORMATO	HORÁRIO	STACK	VERSÃO
5 dias · 4h/dia	19h30 – 23h30 · ao vivo	100% Open-Source · Local-First	OSS v2

OntoLayer

Converge · 8 Passes

AgentSpec

Dark Factory

Golden-Match

Multi-Model Harness

O que há neste documento

01 · O Caso NorthWind Pay EDP	A história, os números, o vilão
02 · A Disciplina — os Rings da Engenharia Agêntica	Os 5 anéis, acesos dia a dia
03 · A Espinha Metodológica	Converge + OntoLayer + AgentSpec
04 · A Stack Open-Source	Local-first, o mapa produção → laptop
05 · Vocabulário de Decomposição	seam → swimlane → leg → task-spec
06 · A Arquitetura Fim a Fim	Legado vs Fábrica Moderna
07 · Dia 1 — O Arqueólogo	Prompt + Context Engineering
08 · Dia 2 — O Tradutor	Harness Engineering
09 · Dia 3 — O Construtor	Loop (semente) · seams → legs
10 · Dia 4 — O Orquestrador	Loop + Eval · o momento AHA
11 · Dia 5 — A Fábrica Autônoma	Dark Factory · 5 Níveis de Shapiro · Multi-Model Harness
12 · A Curva Completa	Visão consolidada dos 5 dias
13 · Formato do Dia & Entregas ao Aluno	Grade de 4h · o arsenal final
14 · Por que este é o Bootcamp GOAT	O que ninguém mais tem

NorthWind Pay EDP

Uma processadora de transações presa a um sistema legado que ninguém entende mais — e a fábrica agêntica que a liberta.

A **NorthWind Pay** processa transações. Cartões passam, PIX cai, boletos liquidam — e a cada ciclo o core cospe **30+ tipos de arquivo cru** que aterrissam num **SFTP**: arquivos `.dat` posicionais e `.txt` em layout proprietário, com COBOL overpunch, PII em claro e cada layout com sua manha.

Entre o SFTP e o banco há um **sistema Java** que ninguém quer mais tocar: ele é quem **lê o arquivo cru, anonimiza a PII** (mascara CPF, tokeniza cartão) e converte para um **CSV sanitizado**, que devolve ao SFTP. Só então **~300 stored procedures PL/pgSQL** processam esse CSV e produzem as conciliações que fecham o dia. *O arquivo cru entra com PII; o Java é a etapa que sanitiza — não o contrário.*

Esse emaranhado nasceu em **3–4 anos, com 8–12 desenvolvedores** que entraram e saíram. As regras de negócio moram dentro das procedures. O Postgres derrete. Cada arquivo novo é cirurgia de risco.

Mas o risco real não é o sistema cair. **É ele funcionar e estar errado.** Um offset trocado, um sinal invertido no overpunch — e nasce *dinheiro plausível-mas-errado*, do tipo que teste não pega e que só aparece três meses depois numa auditoria. **Plausível não é correto.**

A diretoria quer migrar. O orçamento previa 12–18 meses de um time inteiro. Travou na pergunta que ninguém sabe responder: *quem garante que o número do sistema novo bate, centavo por centavo, com o do velho?*

A resposta não é mais gente. É uma fábrica agêntica. E o segredo mais elegante do design: **o sistema legado, com todos os defeitos, é o oráculo** — a fonte da verdade contra a qual cada tradução é validada, centavo por centavo. A missão do **AI-Native Engineer**: construir a fábrica que entende o legado, traduz o arquivo cru e o Java pra Python, ergue a plataforma moderna, e valida cada centavo contra o oráculo — até rodar sozinha.

Os números que tornam isso "impossível na mão"

30+

TIPOS DE ARQUIVO

~300PROCEDURES PL/
PGSQL**3–4 anos**

8–12 DEVS

~42xMAIS RÁPIDO
(BENCHMARK REAL)**~98%**

MENOS CUSTO

Ancorado num sistema real de produção

Este caso é a versão didática do **KurvPay EDP** — um projeto real (Pythian/Kurv) que já roda em produção com 29 de 32 tipos completos, ~42x mais rápido que o T-SQL legado e ~98% de corte de custo, validado golden-match contra o sistema original. O bootcamp destila esse sistema real numa versão 100% open-source que o aluno reproduz no laptop.

Os Rings da Engenharia Agêntica

A disciplina é feita de anéis concêntricos. Cada dia acende um anel — e cada anel externo contém e depende de todos os internos. Ao fim, todos estão acesos e orquestrados.

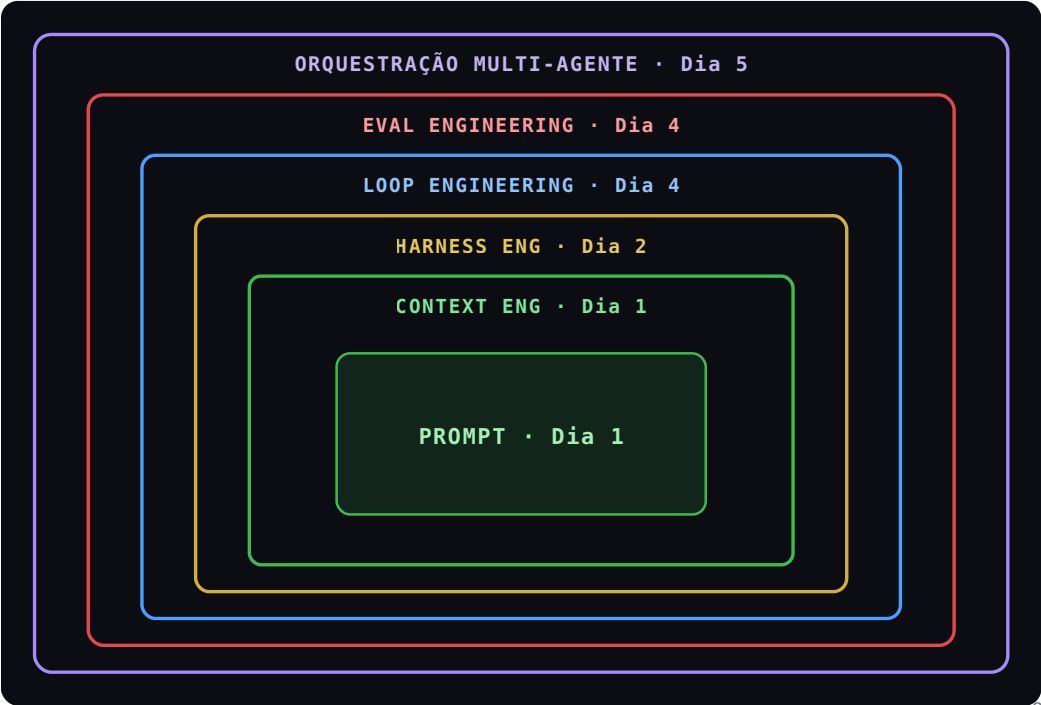


Fig. 1 — Os anéis concêntricos da Engenharia Agêntica, mapeados aos 5 dias do bootcamp.

Dia	Nome	Rings acesos
1	O Arqueólogo	PromptContext Engineering
2	O Tradutor	Harness Engineering
3	O Construtor	Harness aplicadoLoop (semente)
4	O Orquestrador	Loop EngineeringEval Engineering
5	A Fábrica Autônoma	Orquestração Multi-Agente

Converge + os 3 ativos proprietários

O bootcamp roda sobre o **Converge** — o framework de 8 passes que compila intenção em sistema evaluated — e usa três ativos autorais que, juntos, tornam o conteúdo impossível de copiar.

Ativo	O que é	Onde brilha
OntoLayer	Camada de ontologia open-source (crawl → enrich → serve). Transforma DDL ilegível em grafo de conhecimento navegável, com text-to-SQL seguro via sqlglot.	Dia 1 — Context Engineering
AgentSpec	Plugin Claude Code de Spec-Driven Data Engineering: 58 agentes, 31 comandos, 23 KB domains, fluxo de 5 fases + comando <code>/migrate</code> .	Dia 2 — primeiras traduções
Converge	Framework de 8 passes (Intent → Structure → Decompose → Consensus → Fork → Harness → Execute → Loop) + a Dark Factory real do kurv-edp.	Dias 2–5 — espinha e destino

Os 8 Passes do Converge

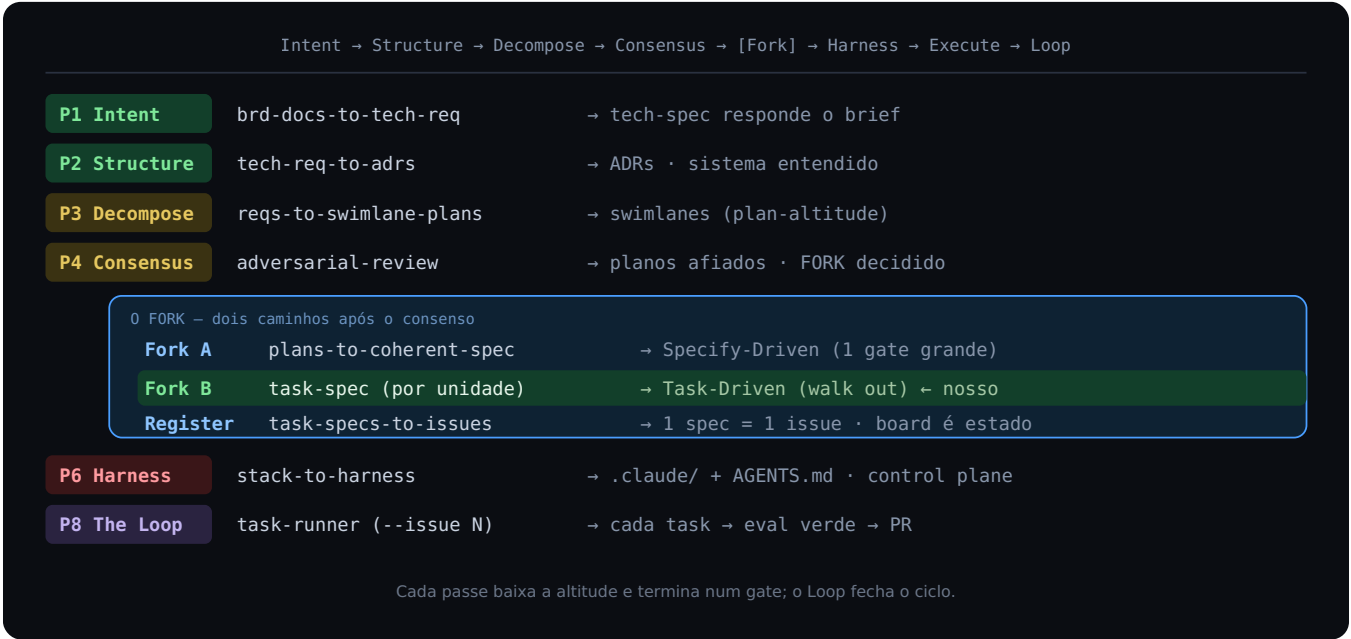


Fig. 2 — As 8 passes do Converge. Cada passe baixa a altitude e termina num gate; o Loop fecha.

Resposta ao medo "63 subagentes esmagam a turma"

O skill de Harness do Converge (`stack-to-harness`) scaffolda o control plane inteiro em minutos — `.claude/` + `AGENTS.md` + KBs + rules. A turma **não monta agentes à mão**: o Converge gera tudo, e o aluno foca em entender e refinar. Curadoria vence abrangência.

Os dois moats (por que é defensável)

Task-Spec = moat de optionality: o eval — não o agente — define "done", então Claude / Codex / Kimi viram slots intercambiáveis. **Harness = moat de qualidade**: os arquivos de control-plane que fazem qualquer agente commodity ter sucesso — construídos ao longo de milhares de horas, não baixáveis.

100% Open-Source, Local-First

A mesma arquitetura de produção do KurvPay real, destilada para o open-source que roda no laptop de qualquer aluno — sem conta cloud, sem custo, com rampa de saída para escalar quando quiser.

Camada	KurvPay real (produção)	Bootcamp (open-source local)
Ingestão	AWS Lambda	Função Python event-driven local
Lakehouse	Databricks Lakeflow	dlt → DuckLake → DuckDB
Transformação	Lakeflow DLT	dbt Core (medallion)
Orquestração	Databricks	Dagster (assets)
Serving	Unity Catalog	FastAPI + MCP
Agêntico	Claude Code + KB + MCP	Claude Code + Codex + Kimi · MCP · Langfuse
IaC / CI-CD	SAM + Azure Pipelines	Terraform + Terragrunt + GitHub Actions

A honestidade que vende

Ensinamos o padrão open-source reproduzível (todo aluno roda) e mostramos o KurvPay real como prova de que o mesmo padrão escala para produção enterprise. O aluno aprende o conceito e vê a evidência de escala. Rampa de saída real: DuckDB → MotherDuck, função local → qualquer nuvem, sem reescrever.

Por que Dagster e não Airflow

Airflow é o padrão de mercado e tem o maior pool de contratação — se você já roda Airflow, não troque. Mas para um projeto **greenfield, agêntico e local-first** como o nosso, Dagster é objetivamente superior: **asset-centric** (cada tabela é um asset com linhagem, casando com a reference architecture do Converge), roda no laptop sem fricção, e é a **superfície mais amigável para agentes de IA**. Escolhemos a ferramenta certa para este caso — não a mais popular.

seam → swimlane → leg → task-spec

Como o trabalho desce da junta natural do sistema até a unidade atômica que um agente executa. Este vocabulário é a base da criação e do aprimoramento das tasks.

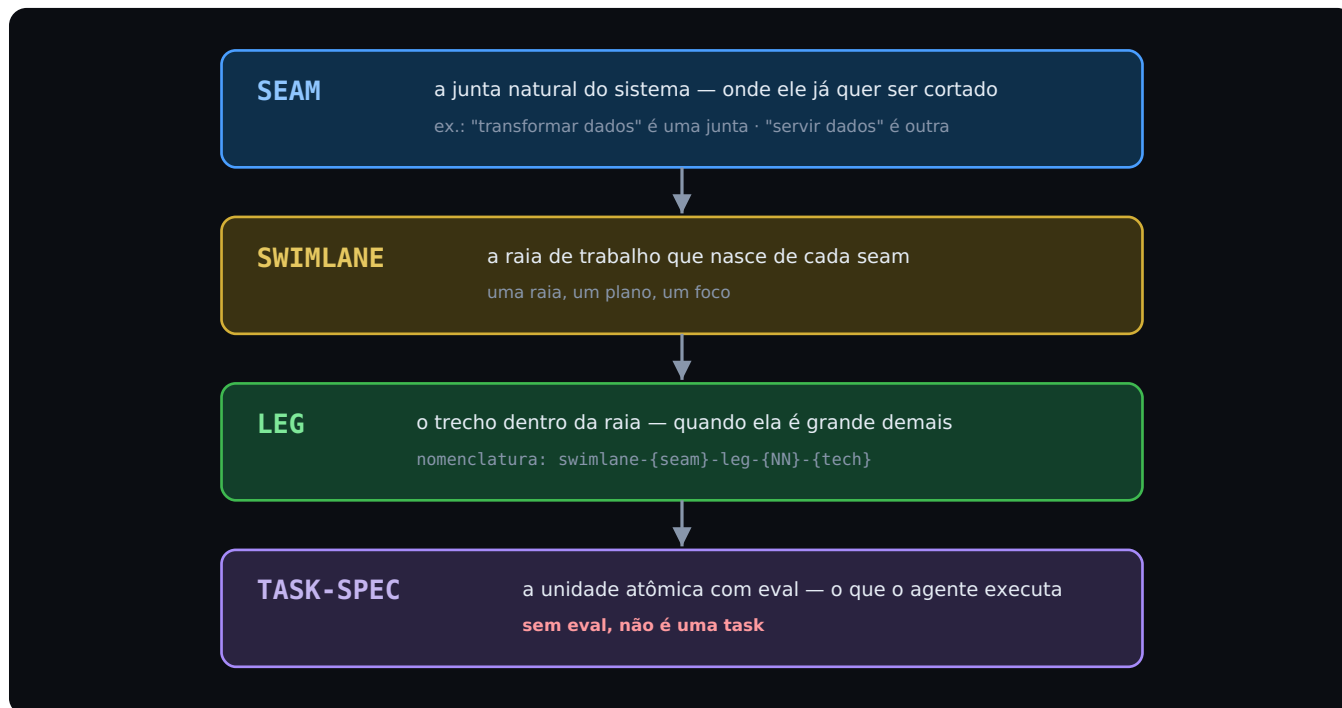


Fig. 3 — A hierarquia de decomposição. Cada nível é mais preciso que o anterior.

O que é um "seam" (costura)

Seam é a junta natural do sistema — como as costuras de uma camisa, que seguem onde as peças de tecido se encontram, nunca no meio de uma peça. No software, o sistema já tem juntas naturais (onde uma responsabilidade termina e outra começa). O Converge corta ali. A costura canônica: **transform** (medallion raw → bronze → silver → gold) versus **serve** (API/MCP sobre o gold).

Por que "leg" (e não "stage")

Numa prova de revezamento, cada nadador faz uma **leg** da raia — exatamente a ideia de subdividir uma swimlane grande em trechos sequenciais. Mantém a metáfora do swimlane coerente e evita colisão com "staging area" / "deploy stage".

Do Legado à Fábrica Moderna

Dois fluxos lado a lado: o emaranhado que a NorthWind Pay tem hoje, e a plataforma que a fábrica agêntica constrói.

O fluxo legado (o que existe hoje)



Fig. 4 — O legado: SFTP → Java → SFTP → Postgres. Regras de negócio presas no código, sem documentação.

O fluxo moderno (o que a fábrica constrói) — 100% open-source

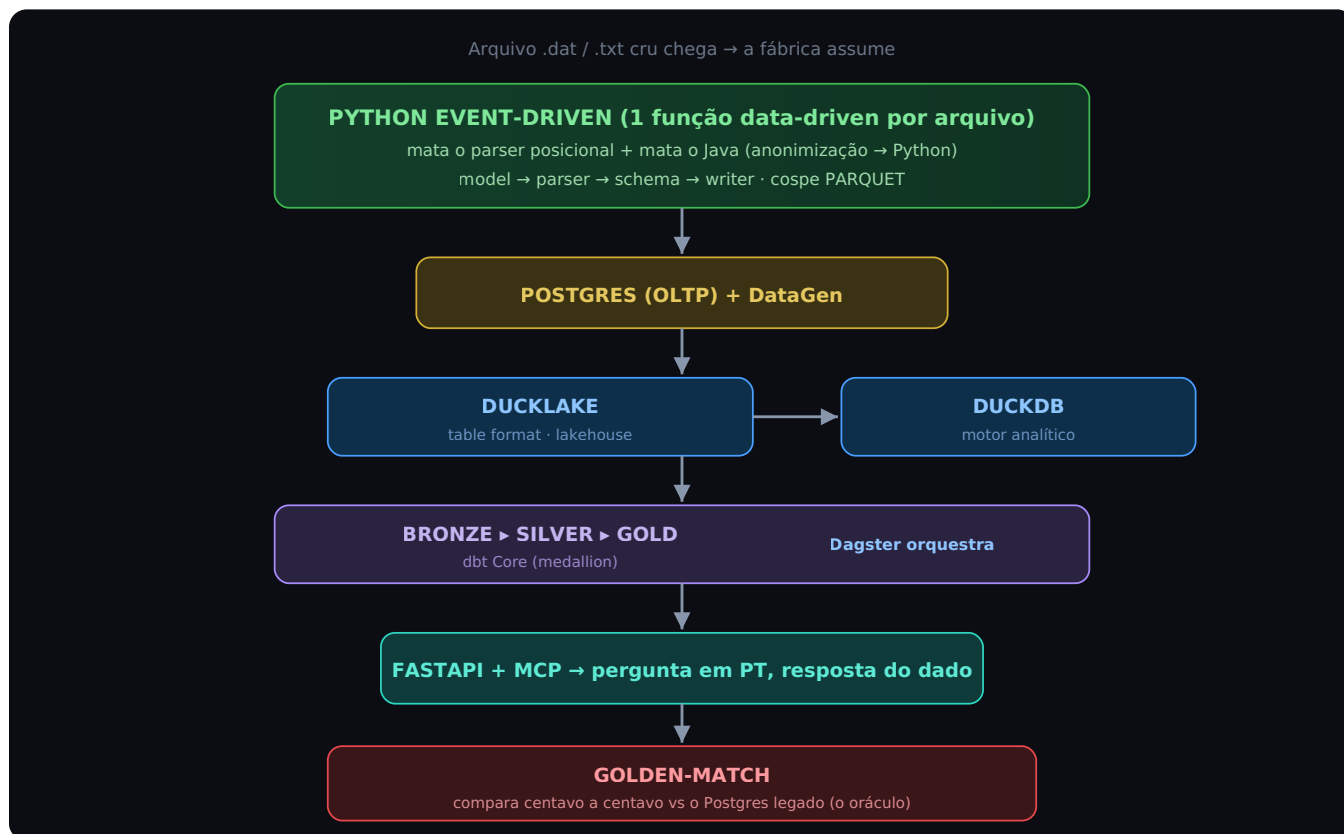


Fig. 5 — A plataforma moderna: event-driven → lakehouse medallion → serving por linguagem natural, validada contra o legado.

O legado é o oráculo — não o inimigo

A fábrica não "confia" na tradução: ela valida cada número contra o sistema legado rodando lado a lado (golden-match). E quando encontra uma divergência, nem sempre o novo está errado — às vezes **é o legado que estava errado o tempo todo**. Essa é a inversão que define o Dia 4.

As personas como agentes da fábrica

O aluno não vira expert nas 5 disciplinas de execução — ele constrói a fábrica que tem **agentes especializados** orquestrados. É assim que uma pessoa faz o trabalho de um time. As cinco disciplinas de execução *descem para os agentes*; o humano *sobe* para o papel que as coordena.

Agente	Persona	O que faz na fábrica
Tradutor	Software Engineer	PL/pgSQL + Java → Python event-driven (4 camadas)
Pipeline	Data Engineer	Ingestão → lakehouse (dlt → DuckLake/DuckDB)
Analytics	Analytics Engineer	dbt medallion bronze/silver/gold
Contexto	AI Engineer	OntoLayer + RAG + MCP serving
Ops	DevOps	Terraform/Terragrunt + CI/CD
Arquiteto	Solutions Architect	Decompõe o problema, desenha as juntas (seams), define o que é "pronto", dirige os evals — o centro de gravidade

A pergunta honesta: o AI-Native Engineer vira "tudo isso"?

Não — e é aí que está a boa notícia. Num horizonte de 2–3 anos, a pessoa não domina as cinco disciplinas de execução à mão. Escrever o SQL, o Terraform, o parser posicional — isso desce para os agentes. O que sobe para o humano é o **julgamento de solução**: qual é o problema real, como cortar nas juntas certas, onde estão os riscos, e o que "pronto" significa de forma verificável.

Por isso o **Solutions Architect não é a sexta ponta igual às outras** — é o eixo para onde o papel migra. O AI-Native Engineer de 2028 se parece menos com um "executor faz-tudo" e mais com um **arquiteto de soluções que também dirige evals**: define a intenção, decompõe, e prova centavo por centavo que a fábrica executou. As cinco personas de execução viram agentes commodity; a arquitetura de solução e o taste de qualidade continuam humanos.

DIA 1 · RING PROMPT + CONTEXT ENGINEERING

O Arqueólogo

Decifrando um sistema que ninguém entende mais

Role: Solutions Architect & AI Engineer

Rings: Prompt + Context Engineering

Converge: Pass 1 (Intent) + Pass 2 (Structure)

⚙️ **Autonomia:** HITL total

A jornada (4h)

- **Injetar o caos no Claude Cowork:** transcripts de reunião, threads de Slack, BRDs, documentação solta do legado.
- **Construir o Second Brain do projeto** — Claude Cowork + Obsidian + Automações via MCP + Schedules. Um organismo vivo que se mantém atualizado sozinho. (*O "wow" do dia: o aluno sai com um segundo cérebro do projeto que se alimenta sozinho.*)
- **Rodar a OntoLayer sobre o Postgres legado:** crawl → enrich → serve. Indexar ~300 procedures + schema, transformando DDL ilegível num grafo navegável. *Sem a base ontológica você não entende como o legado conecta — e é o tema que está explodindo lá fora.*
- **Conectar via MCP** e fazer perguntas de negócio em linguagem natural ao legado.
- **Pass 1 (Intent):** o material vira uma tech-spec que responde o brief.
- **Pass 2 (Structure) — brownfield:** as perguntas à OntoLayer descobrem os fatos (joins reais, grain, o que é "pago"), e **transcripts de reunião endossam** cada decisão → viram os ADRs (fatos + constraints, nunca "como construir").

Entregáveis do Dia 1

- ✓ Second Brain rodando (Cowork + Obsidian + MCP + Schedules)
- ✓ OntoLayer indexando o Postgres, acessível via MCP
- ✓ Tech-spec (Pass 1) + ADRs endossados por transcripts (Pass 2)

Gate: o sistema entende o legado melhor que você.

O Tradutor

Do arquivo cru ao Python — as primeiras vitórias, com rede de segurança

Role: Software Engineer (SWE)

Rings: Harness Engineering

Converge: Pass 3 + Pass 4 + Fork + Pass 6 (Harness)

Autonomia: HITL forte

A jornada (4h)

- **Criar o projeto no Git**, iniciar o Converge fim a fim.
- **Pass 3 (Decompose)**: cortar o trabalho nos *seams* naturais → swimlanes (lê os ADRs do Dia 1).
- **Pass 4 (Consensus)**: um adversário (Codex) ataca os planos — refuta, aponta drift, **decide o Fork**.
- **O skill de Harness scaffolda tudo em minutos**: `stack-to-harness` gera `.claude/` + `AGENTS.md` + KBs + rules. A turma foca em entender e refinar.
- **As duas lanes do Converge**:
 - Specify-Driven (AgentSpec)**: `/agentspec:brainstorm` → `define` → `design` → `build` → `ship` e `/agentspec:migrate` geram as primeiras traduções guiadas — a porta de entrada, vitórias rápidas.
 - Task-Driven (introdução)**: como as traduções descem para task-specs atômicos — a semente do Dark Factory.
- **O padrão de 4 camadas por tipo** (destilado do KurvPay): `model` → `parser` → `schema` → `writer`. Ensina UM tipo à fundo — money é `Decimal` nunca `float`, COBOL overpunch, o -1 do 1-indexed SQL → 0-indexed Python.
- **Nasce a função event-driven Python**: recebe o `.dat/.txt` cru → anonimiza a PII (mata o Java) → parseia o layout posicional (mata os PL/pgSQL de ingestão) → cospe **Parquet**.
- **HITL forte**: valida cada tradução, constrói confiança. *Sem Loop ainda — de propósito.*

Entregáveis do Dia 2

- ✓ Projeto Git com Converge (tech-spec, ADRs, plans, consensus, fork)
- ✓ Harness completo scaffoldado pelo skill (o quality moat)
- ✓ AgentSpec rodando (fluxo Specify-Driven)
- ✓ Primeiros tipos traduzidos no padrão 4-camadas → Parquet
- ✓ Primeiros task-specs atômicos esboçados

Gate: o sistema traduz legado e roda isolado, sem risco.

DIA 3

O Construtor

Erguendo a plataforma moderna, leg por leg

Role: Data Engineer & Analytics Engineer

Rings: Harness aplicado + Loop (semente)

Converge: Pass 3 (seams → swimlanes → legs) + Pass 5B (Tasking)

⚙️ **Autonomia:** HITL saindo

A jornada (4h)

- **OLTP já pronto (economia de tempo):** DataGen + Postgres + ~300 procedures vêm prontos como "o sistema atual" — o aluno foca em migrar, não em montar o legado.
- **Os seams cortados em swimlanes, e as swimlanes grandes quebradas em legs** (granularidade fina para criar e aprimorar as tasks separadamente).
- **Task-Spec em profundidade:** com tempo, explica-se tudo sobre Task-Spec — o primitivo que é o ganho para chegar na Dark Factory (o eval define done, o agente vira commodity).
- **Construção com HITL, não Loop ainda:** gerar as tasks e pedir para o **Claude Code Workflows** executar, OU fazer 1 a 1 validando — saindo do HITL aos poucos, construindo confiança.

Os seams, swimlanes e legs do pipeline

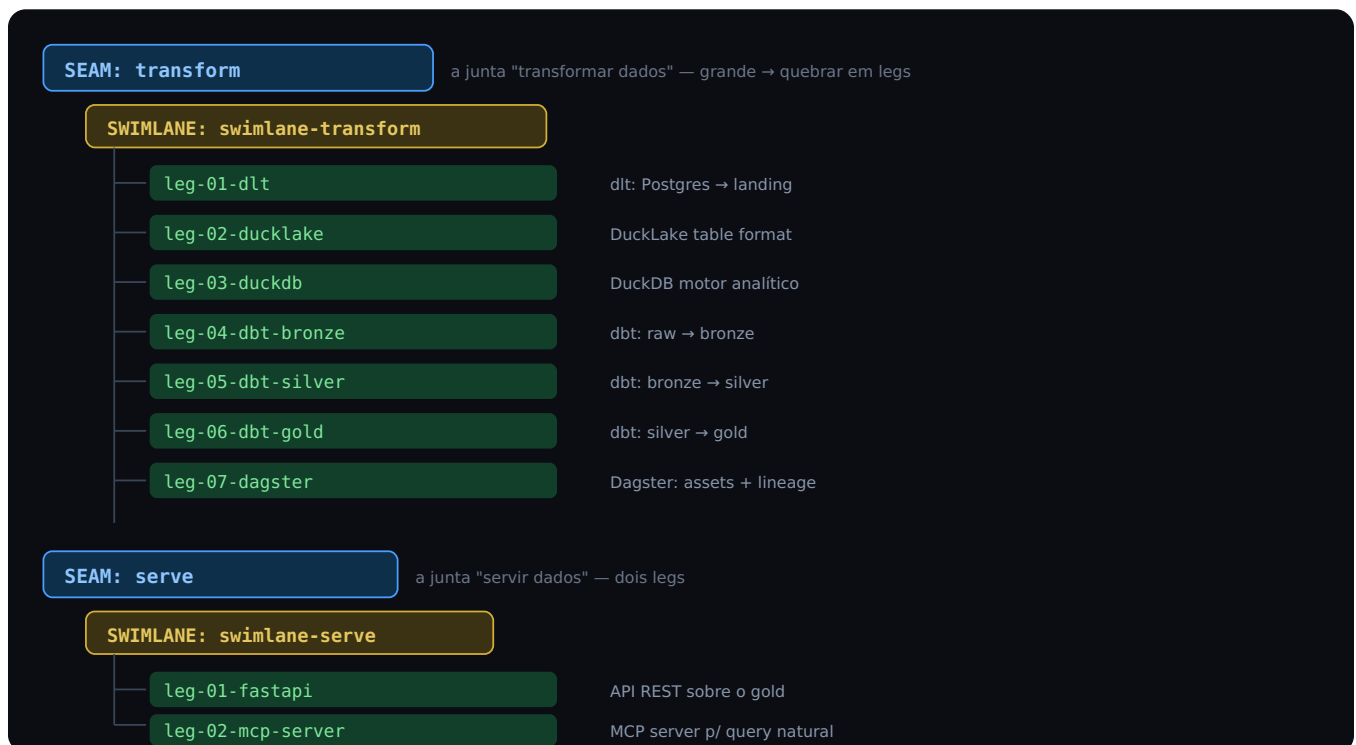


Fig. 6 — Decomposição: 2 seams → 2 swimlanes → 9 legs. Cada leg vira um conjunto de task-specs.

Por que Dagster e não Airflow

Asset-centric casa com a reference architecture do Converge (cada tabela é um asset com linhagem), roda local sem fricção, e é a superfície mais amigável para agentes. Airflow é o padrão de mercado — mas para greenfield agêntico local-first, Dagster é superior.

Entregáveis do Dia 3

- ✓ Pipeline local fim a fim: bronze → silver → gold entregando dados
- ✓ Swimlanes quebradas em legs nomeados e rastreáveis
- ✓ FastAPI + MCP server (dois legs da serve) sobre o gold
- ✓ Conectar o Claude Code e fazer perguntas naturais sobre o dado
- ✓ Biblioteca de Task-Specs (por leg) pronta para virar Loop

Gate: o sistema constrói o pipeline inteiro sozinho — ainda com você validando.

DIA 4

O Orquestrador

Tirando as mãos do volante — o Loop assume, e a fábrica indicia o oráculo

Role: AI-Native Engineer & DevOps

Rings: Loop + Eval Engineering

Converge: Pass 8 (The Loop, fechado) + Manager (CI/CD)

Autonomia: Loop total

A jornada (4h) — passo a passo

1. **Ativar o Loop (task-runner):** pela primeira vez sem HITL. Pega um issue (`--issue N`), lê task-spec + ADRs + harness, cria branch, escreve código, roda o eval. RED → alimenta a falha de volta e revisa (loop local até verde). GREEN → abre PR que fecha o issue. *Convergência é o eval verde, nunca número de tentativas.*
2. **Gestão profissional com Linear:** cada task-spec vira 1 issue no Linear (via MCP). `blocked-by` carrega o grafo de dependências. Só o loop escreve estado; só eval verde fecha issue — **o board nunca mente**. O Linear vira o painel executivo: dá para mostrar ao cliente/gestão o progresso da fábrica em tempo real. *Não é "script rodando", é uma operação de engenharia visível.*
3. **Kanban-as-EM:** o Linear vira o Engineering Manager — decide o que está pronto, bloqueado, escalado.
4. **Manager = CI/CD (GitHub Actions):** scheduler decide qual issue roda, fan-out paralelo, observa PRs, desbloqueia dependentes.
5. **The Wall:** Gerador e Validador nunca compartilham contexto (anti-sycophancy).
6. **Cross-cutting concerns:** Security (RBAC + anonimização/PII + prompt injection) · Cost (model tiering: Kimi barato p/ volume, Claude caro p/ raciocínio) · Observability (Langfuse + OpenTelemetry).
7. **Tasks já em brownfield:** injetar novos requisitos e ver a fábrica gerar/controlar tasks sozinha.
8. **Deploy em dev:** Terraform + Terragrunt + GitHub Actions.

O MOMENTO AHA — Golden-Match indicia o oráculo

Eval-Driven Development: o golden-match compara a procedure PL/pgSQL antiga vs o Python novo, lado a lado, centavo por centavo.

O erro plantado: no sistema Java legado existe um **erro silencioso plantado** — um overpunch invertido num campo de valor, ou um offset trocado que soma centavos errados há anos. O legado sempre "funcionou". Ninguém nunca notou.

A fábrica descobre sozinha: o golden-match acusa a divergência. A turma investiga e descobre que **o legado — a suposta fonte da verdade — estava errado o tempo todo**. A fábrica indicou o oráculo.

"O golden-match existe porque um número errado de dinheiro passa despercebido." É o momento que a turma nunca esquece — entendem na pele por que Eval Engineering não é opcional em sistema financeiro.

O Loop de execução (Pass 8)

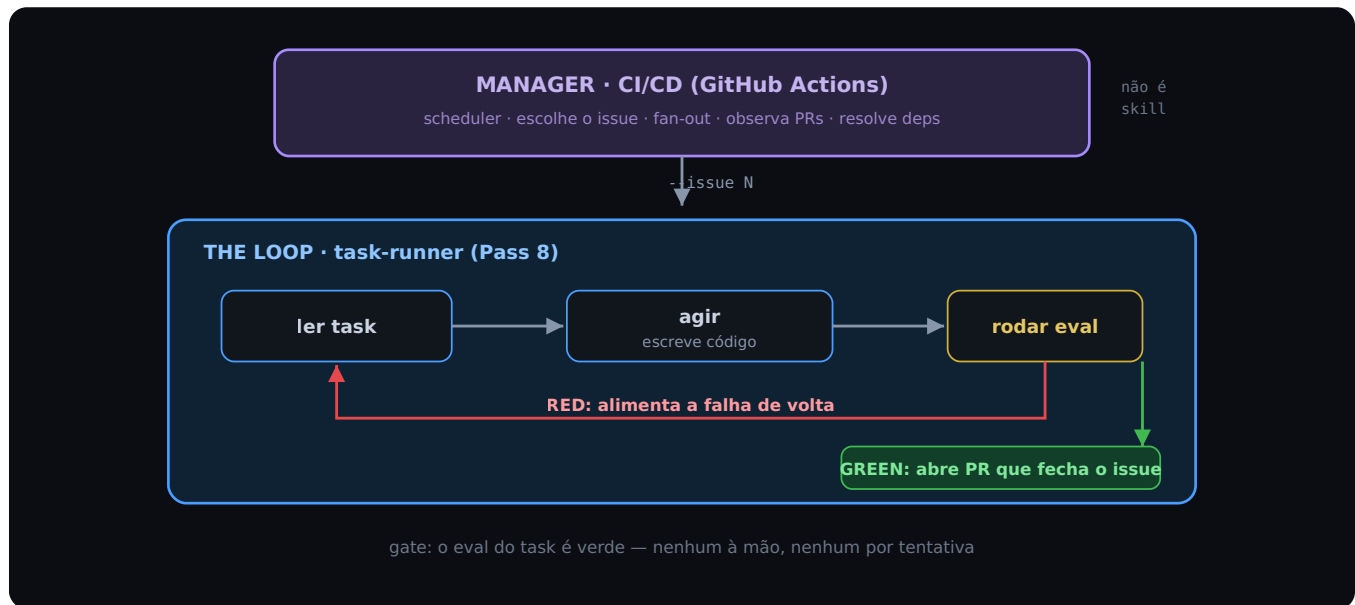


Fig. 7 — O Loop fecha o ciclo: um issue entra, código sai, o eval decide. O Dark Factory, um issue de cada vez.

Entregáveis do Dia 4

- ✓ Loop fechado: issues fechando por eval verde, sem mão humana
- ✓ Linear como control plane profissional (painel executivo da fábrica)
- ✓ Manager (CI/CD) dispatchando + fan-out paralelo
- ✓ Golden-match indiciando o erro plantado no legado (o AHA)
- ✓ Cross-cutting aplicado (Security + Cost + Observability)
- ✓ Deploy em dev (Terraform/Terragrunt)

Gate: a fábrica valida a própria migração — e prova que o legado estava errado.

DIA 5

A Fábrica Autônoma

Continuous Improvement & Dark Factory — a fábrica que roda e se conserta sozinha

Role: AI-Native Engineer (todas as roles integradas)

Rings: Orquestração Multi-Agente

Converge: Loop em produção + brownfield + self-healing

⚙️ **Autonomia:** Dark Factory

A jornada (4h)

- **Mostrar o processo fim a fim completo** — do brief ao deploy, cada passo eval-verified.
- **O MVP provado:** ao longo dos 4 dias a fábrica processou **5 tipos diferentes** de arquivo (o MVP que prova o padrão). Agora vem o teste final.
- **O momento GOAT — Dark Factory com 1 tipo NOVO ao vivo:** joga um **6º tipo, nunca visto**, na fila. A fábrica dispara sozinha: scaffold → entende via OntoLayer → gera parser (4 camadas) → testa → golden-match contra o legado → deploy. **Ninguém escreve código.** A turma assiste a fábrica fazer onboarding de um tipo novo sozinha.
- **Self-healing da fábrica (o flywheel):** cada tipo processado colhe **lições** (memory) que a fábrica lê primeiro no próximo. Quando uma lição recorre, vira **skill** — a fábrica melhorando a fábrica. (*Destilado do KurvPay real: 55 lições viraram 4 skills — money, overpunch, positional, reconciliation.*)
- **Ground truth check em produção:** pedir o resultado final do Postgres legado e comparar com o output moderno — golden-match de produção.
- **Continuous Improvement (brownfield):** o sistema virou brownfield. Nova demanda → task-spec → issue no Linear → loop → PR. O ciclo completo de evolução contínua.

A Dark Factory — pipeline gated + flywheel de lições

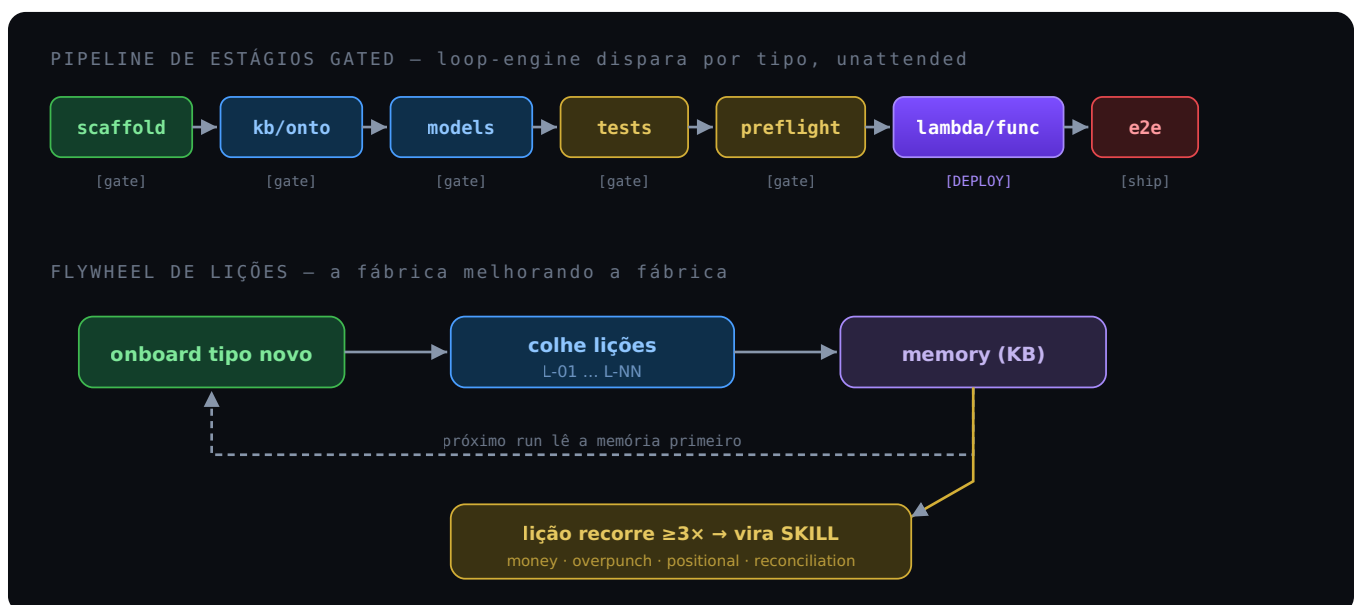


Fig. 8 — A Dark Factory: pipeline de estágios gated + o flywheel que faz a fábrica ficar mais rápida e segura a cada tipo.

A honestidade que separa você do hype

O loop converge para evals verdes, não para outcomes corretos. Se a spec está errada, o código implementa fielmente o erro. Por isso o Pass 1 (o gate força a spec a responder o brief) e o Pass 4 (o adversário ataca antes de qualquer código) existem. O Dark Factory automatiza a construção — **não o julgamento de se a spec estava certa**. Essa honestidade intelectual é o que te separa dos vendedores de hype.

Os 5 Níveis de Dan Shapiro — de autocomplete apimentado à Dark Factory

Dan Shapiro (jan/2026) mapeou a automação de código nos mesmos 5 níveis que a NHTSA usou para carros autônomos. É a régua que situa *onde cada engenheiro está* — e para onde a fábrica leva o aluno.

0

VOLVO DOS PAIS

Manual

Nenhum caractere chega ao disco sem sua aprovação. IA como buscador turbinado ou um tab ocasional. O código é inegavelmente seu — trabalho braçal num mundo deflacionário.

1

LANE-KEEPING

Tarefas discretas delegadas

"Escreve um teste unitário pra isso." "Adiciona um docstring." Você escreve o que importa e passa o chato pro estagiário-IA. Speedups, mas o trabalho não mudou — você ainda anda na velocidade que digita.

2

AUTOPILOT

Pairing em flow state

Onde vivem 90% dos devs "AI-native" hoje. Você parecia com a IA como um colega, num estado de fluxo, mais produtivo do que nunca. **O perigo:** o L2 (e todo nível depois dele) dá a sensação de que você terminou — mas não terminou.

3

WAYMO C/
MOTORISTA

Você virou gerente

Você não é mais o dev sênior — isso é trabalho da IA. Você é o humano no loop, com várias abas rodando. Seus dias são revisar código. Sua vida vira diffs. **Quase todo mundo estaciona aqui.**

4

ROBOTAXI

Você virou PM

Você escreve uma spec, discute a spec, cria skills (quase sempre no Claude Code), planeja, revisa planos — e vai embora por 12h para depois checar se os testes passaram. É onde o autor do conceito diz estar.

5

DARK FACTORY

A fábrica autônoma ← estamos mirando aqui

Não é mais um carro. O processo de software não é mais um processo de software — é uma caixa-preta que transforma specs em software. "Dark" como a Fanuc Dark Factory: uma fábrica onde humanos não são nem necessários nem bem-vindos. Times de menos de 5 pessoas fazendo o quase inacreditável.

Onde este bootcamp mira

A maioria do mercado empaca no **Nível 3** (virar revisor de diffs) e sente que piorou. Este bootcamp existe para levar o aluno do L3 ao **L5** — construindo a Dark Factory do NorthWind Pay EDP com as próprias mãos. A régua de Shapiro é o mapa; o UC é o percurso.

Multi-Model Harness — cada modelo no seu papel

A fábrica não roda num modelo só. O padrão validado em produção (Omnigent, oh-my-* e o cross-vendor review) separa papéis por provedor — e o **revisor é sempre de um provedor diferente de quem escreveu**, para quebrar o viés de concordar consigo mesmo (o "The Wall" do Ring de Eval). Task-Spec é o que torna isso possível: o eval define "pronto", então o executor vira um slot commodity.

Claude Code

Orchestrator. Decompõe a tarefa, roteia para os sub-agentes, sintetiza. Mantém autoridade de escrita/merge e o contexto enxuto. Nunca escreve o grosso do código — coordena.

Codex

Reviewer. Revisa correção, segurança, estilo e performance. Como é de provedor diferente do executor, o review é cross-vendor — a mitigação estrutural contra sycophancy. Devolve PASS / REVISE / REJECT.

Kimi

Executor. Escreve o código, cria os testes, roda a verificação numa worktree isolada. Lane advisory até o review passar.

GLM

Executor. Segundo executor em paralelo — permite fan-out de traduções independentes (ex.: vários tipos de arquivo ao mesmo tempo), cada um na sua worktree, reconciliados no merge.

Por que isso é o moat de optionality na prática

Quando o eval (não o agente) define o "pronto", o executor é intercambiável: Kimi hoje, GLM amanhã, o que for mais barato/capaz na semana que vem. O orquestrador (Claude Code) e o revisor cross-vendor (Codex) formam o control-plane estável; os executores são commodity. É exatamente o padrão `Orchestrator → Executor → cross-vendor Review → merge` que a fábrica usa por baixo.

Entregáveis do Dia 5

- ✓ Dark Factory autônoma fazendo onboarding de um tipo novo ao vivo
- ✓ Self-healing/flywheel demonstrado (lições → skills)
- ✓ Deploy em produção (Terraform/Terragrunt)
- ✓ Ground truth validado contra o legado
- ✓ Ciclo de continuous improvement (brownfield)
- ✓ Capstone apresentado (Demo Day)

Entrega final: um sistema agêntico que migra legado → moderno sozinho, se valida e se conserta. Você virou AI-Native Engineer.

A Curva Completa dos 5 Dias

A progressão inteira num só quadro — nome, rings, role, passes do Converge, autonomia e o momento-chave de cada dia.

Dia	Nome	Rings	Role	Converge	Autonomia	Momento-chave
1	O Arqueólogo	Prompt+Context	Sol. Arch & AI Eng	P1, P2	HITL total	Second Brain + OntoLayer
2	O Tradutor	Harness	SWE	P3, P4, Fork, P6	HITL forte	Harness em minutos + 4 camadas
3	O Construtor	Harness + Loop (semente)	Data & Analytics	P3 seams → legs, P5B	HITL saindo	Swimlanes → legs
4	O Orquestrador	Loop + Eval	AI-Native & DevOps	P8, Manager	Loop total	Golden-match indicia o oráculo
5	A Fábrica Autônoma	Orquestração	AI-Native (todas)	Produção + brownfield	Dark Factory	Tipo novo ao vivo + self-healing

A curva de autonomia (o coração pedagógico)



Fig. 9 — A curva de autonomia. Cada dia remove um pouco mais do humano do caminho de execução.

Cobertura de roles — todas atendidas

Role	Dia principal	Reforço
Solutions Architect	Dia 1	Dia 4
AI Engineer	Dia 1, Dia 4	Dia 3
Software Engineer (SWE)	Dia 2	Dia 5
Data Engineer	Dia 3	Dia 5
Analytics Engineer	Dia 3	Dia 4
DevOps	Dia 4	Dia 5
AI-Native Engineer (integra todas)	Dia 4, Dia 5	todos

Formato do Dia & Entregas ao Aluno

Grade de cada dia (4h · 19h30 – 23h30)

Bloco	Horário	O quê
Abertura + recap	19h30 – 19h50	Onde paramos, o que a fábrica já faz
Conceito + demo do artefato	19h50 – 20h50	Mostra o artefato semi-pronto e explica
Mão na massa guiada	20h50 – 21h40	Turma constrói junto
Intervalo	21h40 – 21h50	—
Mão na massa profunda	21h50 – 23h00	Construção do entregável do dia
Fechamento + gate check	23h00 – 23h30	Validar o gate, prévia do dia seguinte

Artefatos semi-prontos (para caber em 4h)

Cada dia começa com um "estado inicial" pronto, para a turma não perder tempo com setup:

Dia	Artefato semi-pronto
1	OntoLayer instalável + Postgres legado populado + transcripts de reunião prontos
2	Harness base (.claude/ + AGENTS.md) esboçado + AgentSpec instalado
3	DataGen + Postgres + ~300 procedures prontas (o OLTP "atual")
4	Task-specs do Dia 3 já registrados como issues no Linear + GitHub Actions configurado
5	Arquivo-surpresa (6º tipo) preparado + ambiente de prod (Terraform) pré-provisionado

O que o aluno leva (o arsenal)

Arsenal do AI-Native Engineer

- ✓ O Converge aplicado ponta a ponta (as 8 passes na prática)
- ✓ A OntoLayer rodando sobre um Postgres real
- ✓ O Second Brain do projeto (Cowork + Obsidian + MCP + Schedules)
- ✓ A fábrica open-source completa (GitHub) migrando 5 tipos + onboarding do 6º
- ✓ A biblioteca de Task-Specs por leg + o harness scaffoldado
- ✓ Certificado "AI-Native Engineer — Nível Produção"

O que ninguém mais tem

1. Baseado num sistema real em produção

KurvPay: 29/32 tipos completos, ~42x mais rápido, ~98% de corte de custo, golden-match validado. Não é teoria — é a cicatriz de um projeto real, destilada em conteúdo.

2. Três ativos proprietários seus

OntoLayer (ontologia), AgentSpec (spec-driven) e Converge (framework + Dark Factory). Ninguém no Brasil ensina com este arsenal — é impossível de copiar.

3. Um caso único que evolui

Do primeiro spec à Dark Factory que se conserta. O aluno sente o sistema ganhar autonomia dia a dia — não assiste tópicos soltos.

4. O momento AHA do erro plantado

A fábrica prova que o legado — a suposta fonte da verdade — estava errado o tempo todo. É o instante que a turma nunca esquece.

5. 100% open-source local

Todo aluno reproduz no laptop, sem cloud, sem custo — com rampa de saída real para escalar.

6. Honestidade de engenheiro sênior

Você ensina os limites (o loop converge para evals, não para a verdade). É o que separa autoridade técnica de venda de hype.

A promessa, numa frase:

Em cinco dias, você não vai aprender a usar agentes — vai **construir um sistema agêntico completo, do zero à produção**, que migra uma plataforma financeira legada sozinho, se valida contra o passado e se conserta para o futuro. Quando acabar, você não terá assistido a um curso. Terá construído o futuro da engenharia.